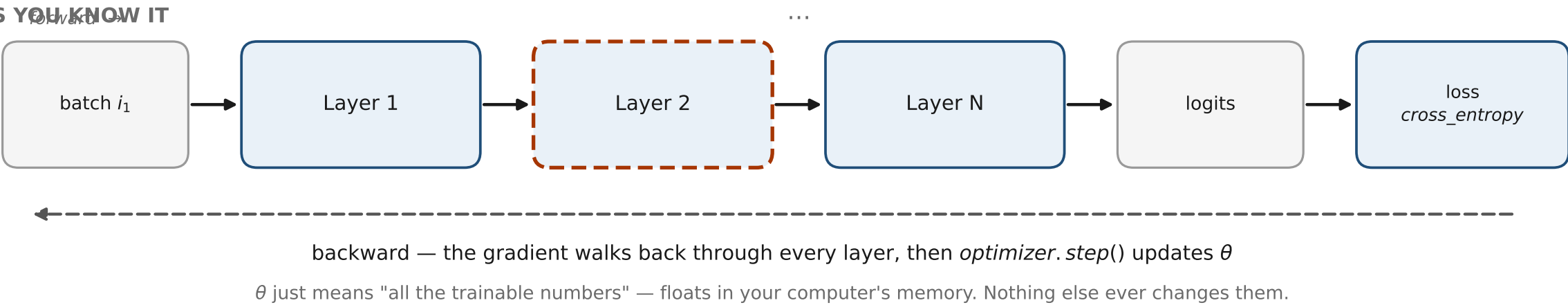
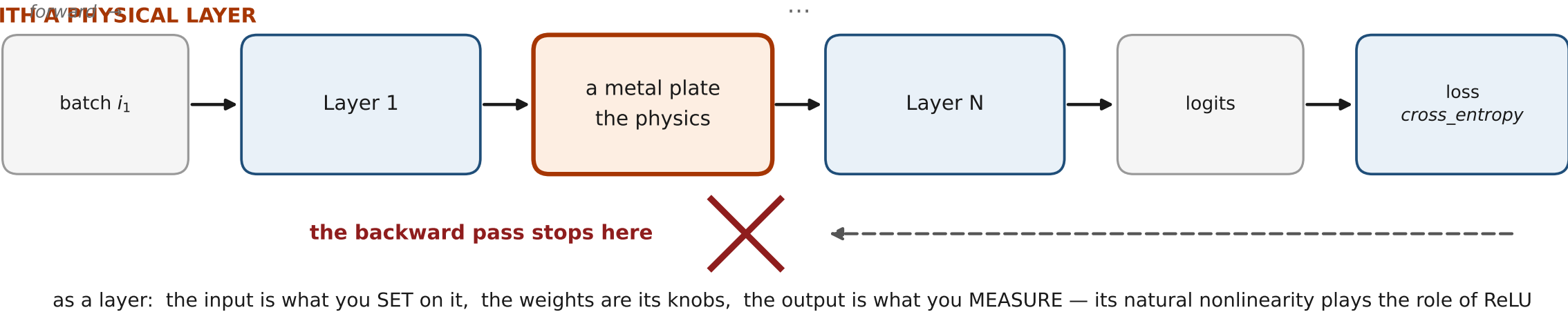


a transformer, or any network — the boxes stand for a whole stack. Only one of them changes.

AS YOU KNOW IT



WITH A PHYSICAL LAYER



UNCHANGED

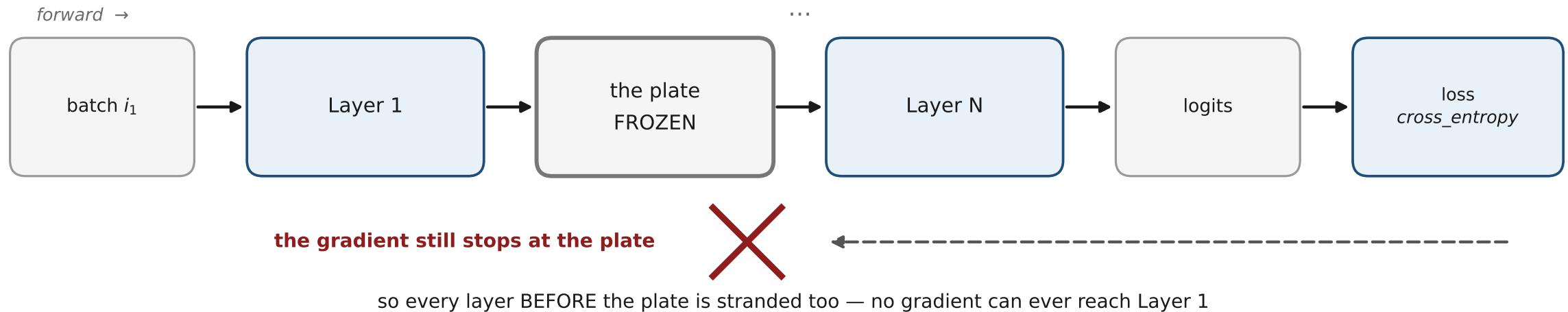
- ✓ the data loading, the other layers, the logits
- ✓ the loss — still `F.cross_entropy( $\hat{y}$ , label)`
- ✓ the optimizer — still Adam
- ✓  $\theta$  — still ordinary floats on your computer

BROKEN

- ✗ the plate has no source code
- ✗ autograd recorded nothing on the way through
- ✗ there is no derivative to walk back
- ✗ so this layer cannot be trained

That single broken arrow is the entire problem. Four ways to attack it — A, B, C, D — then three experiments.

*freeze the knobs and let the digital layers do all the learning. The simplest idea, and it half-works.*



## why it half-works

- a fixed, random, nonlinear mixing of the data is genuinely useful
- put the physics FIRST and train only a digital readout after it — this is reservoir computing

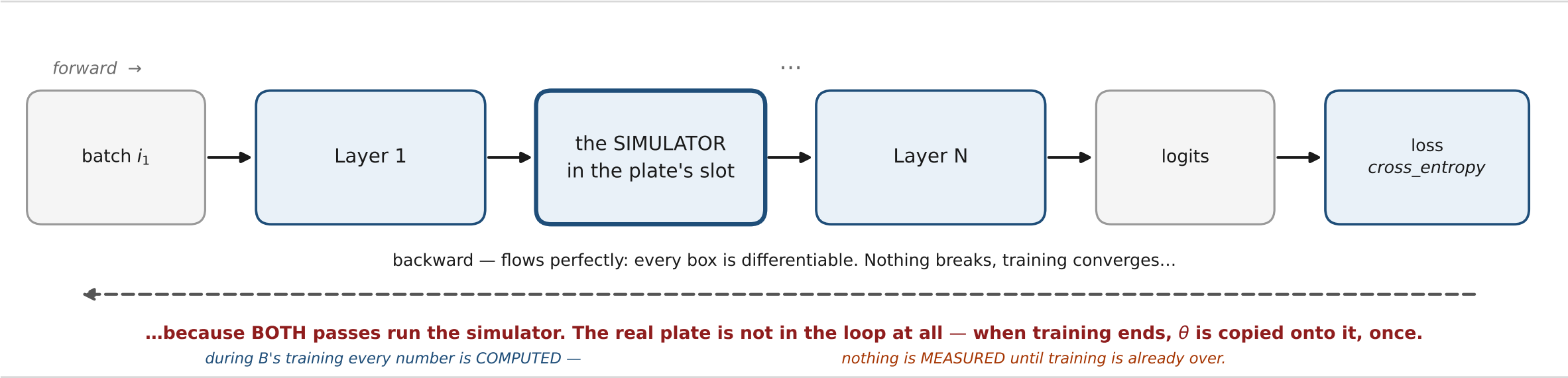
## why it is not enough

- the physics never adapts to your task
- all learning is pushed into digital layers — the exact compute the device was built to avoid
- millions of knobs sit at random settings, unused

**✗ VERDICT: usable, but it wastes the device — the physical layer is a random mixer, not a trained computer.**

*the knobs themselves have to learn. Next: replace the plate with a model of it.*

then copy the finished weights onto the plate, once. The plate leaves the loop entirely — everything flows, and that is the trap.

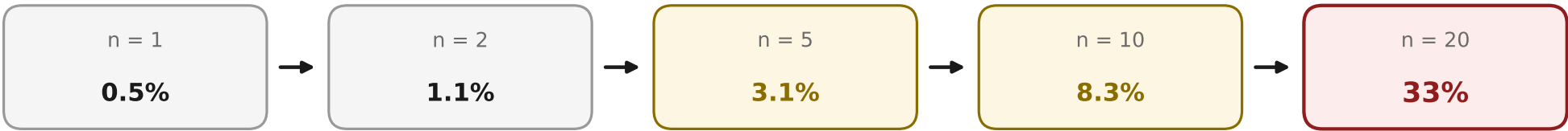


the simulator, properly:

$f_{model}(x, \theta)$  — differentiable torch code that predicts the plate: the physics written out, or a small net FITTED to the device — trained once, beforehand, on thousands of measured  $(x, \theta) \rightarrow y$  pairs. In that tiny side-training the device's own outputs are the TARGETS.

the paper's “differentiable digital model”. Either way it is slightly *WRONG* — constants off, effects unmodelled. Shrinkable, never closable.

and slightly wrong is fatal at depth — suppose it is off by just 0.5%:

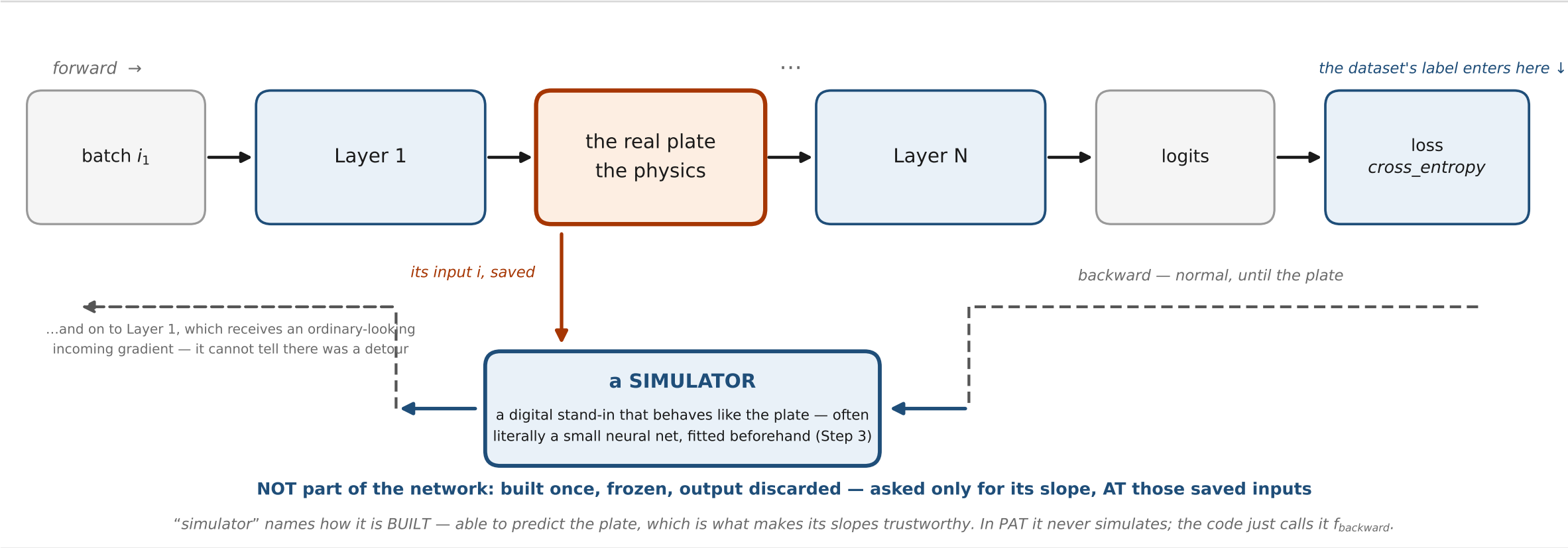


errors COMPOUND — every layer works on the previous layer's already-wrong answer. The weights land tuned for a machine that does not exist.

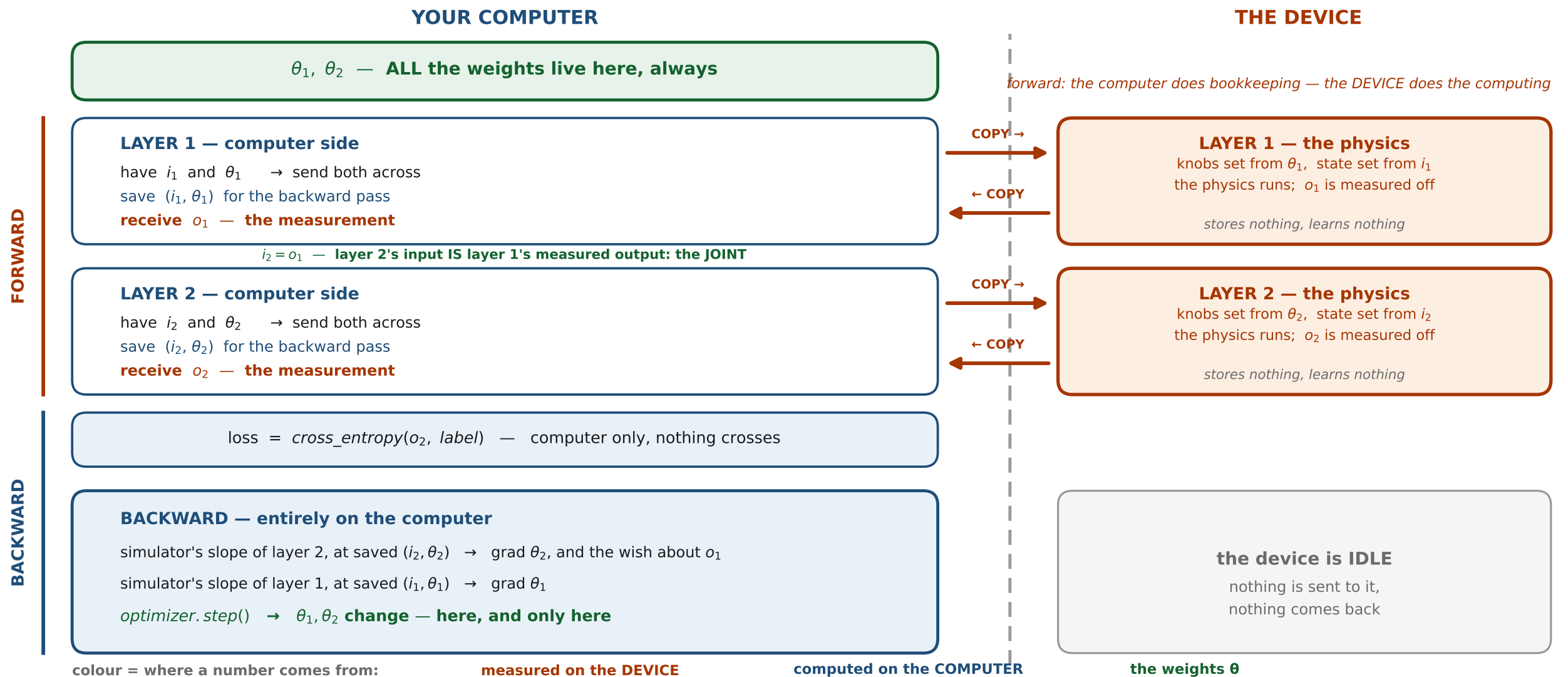
✗ **VERDICT: fails SILENTLY** — from inside the simulator everything looks excellent. The drop only appears on the hardware, after training is done.

**PAT — next page — keeps this exact simulator, demoted to one job: B trusted its OUTPUTS, PAT uses only its SLOPES.**

forward runs the REAL plate. When the gradient reaches the plate on the way back, it detours through a simulator.



*two physical layers, one training step. Every crossing is plain numbers; everything else stays put.*



**Copied over, per layer, per batch:**

→  $\theta_l$ , written into the dials + the layer's input state

← one measured output vector

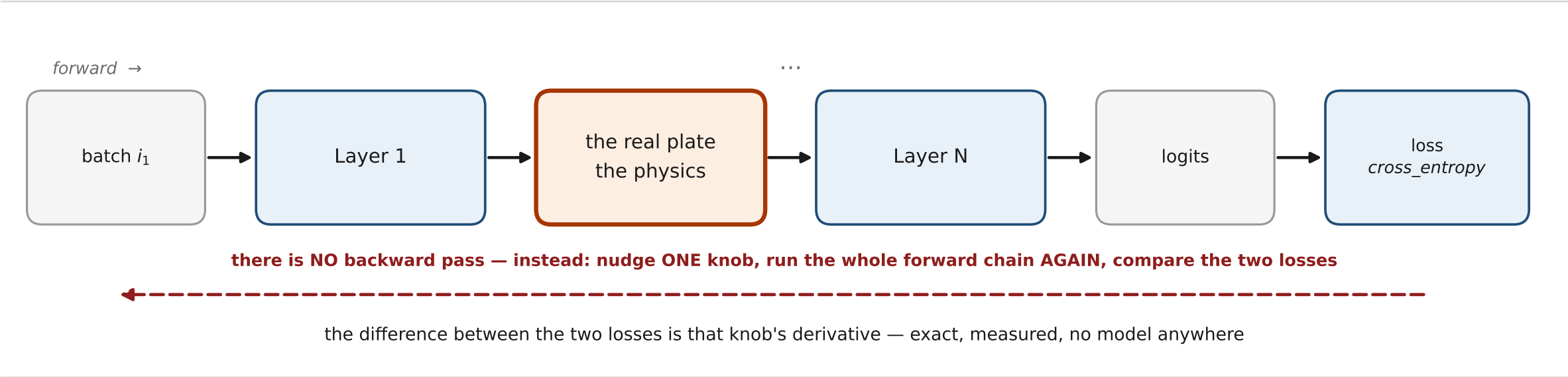
**never copied:** the label · the loss · any gradient · the simulator · autograd

next batch, the same  $\theta$  is written to the dials AGAIN — the device kept nothing, so nothing needs copying back.

**why TWO layers? PAT's advantage lives at the joint: layer 2's slope is taken at the MEASURED  $\phi_1$ , not the simulator's guess of it.**

**One layer has no joint — and B would do almost as well.**

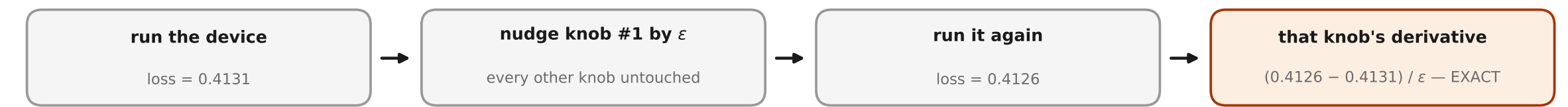
no simulator, no assumptions. Honest, exact — and seeing why it is unaffordable is what makes PAT click.



why it takes two runs per knob:

one run gives a VALUE — “with these knobs, this came out”. Training needs a SLOPE, per knob — “how would the loss move if knob #4,192 moved?”

a slope cannot be read off a single point. Each one costs a fresh run of the entire chain:



...now repeat, from the top, for knob #2. And #3. And every one of the millions — for EVERY training step.

✗ VERDICT: perfectly honest, completely unaffordable — one hardware run per knob, per step.

note the colour: in D even the SLOPE is orange — measured. That is its honesty, and its cost.

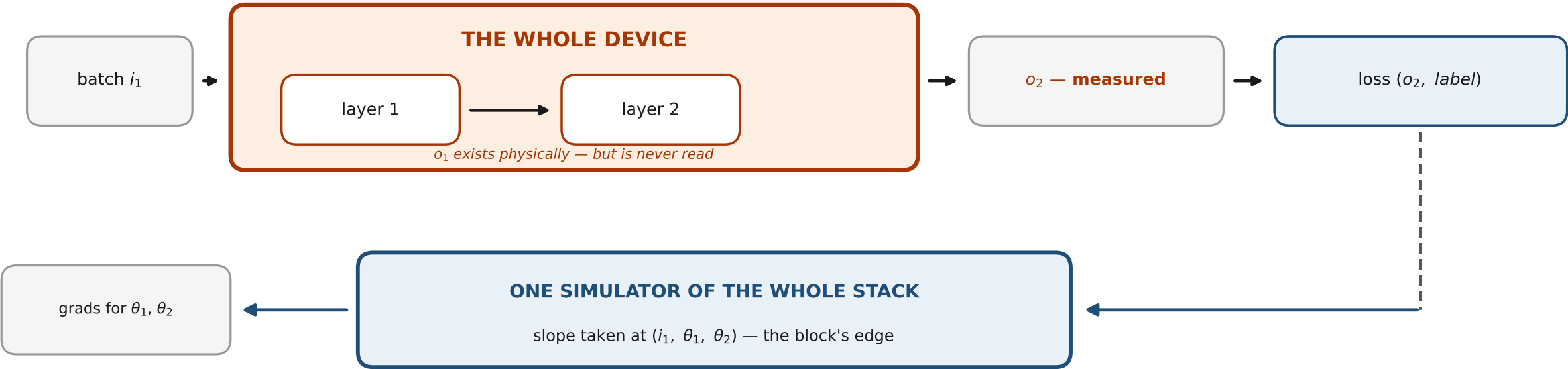
A wasted the device.    B trusted a wrong simulator.    D cannot afford exactness.

PAT takes D's instinct — trust only real measurements — and B's simulator, demoted to slopes.

that is PAT — Step 4. One refinement question remains (next page), then the experiments.

the natural refinement question: why per-layer at all? Run the device end to end, measure once, use one big simulator.

FORWARD — real, end to end, measured ONCE at the exit



but inside, its slope must chain ITSELF:  $J_2(\tilde{o}_1) \cdot J_1(i_1)$  —  $\tilde{o}_1$  is its OWN guess. Compounding returns, inside the block.

PROS

- the forward pass and the loss are still REAL —  
B's silent failure cannot happen
- only the final output needs measuring — the ONLY option  
when intermediates are physically unreachable

CONS

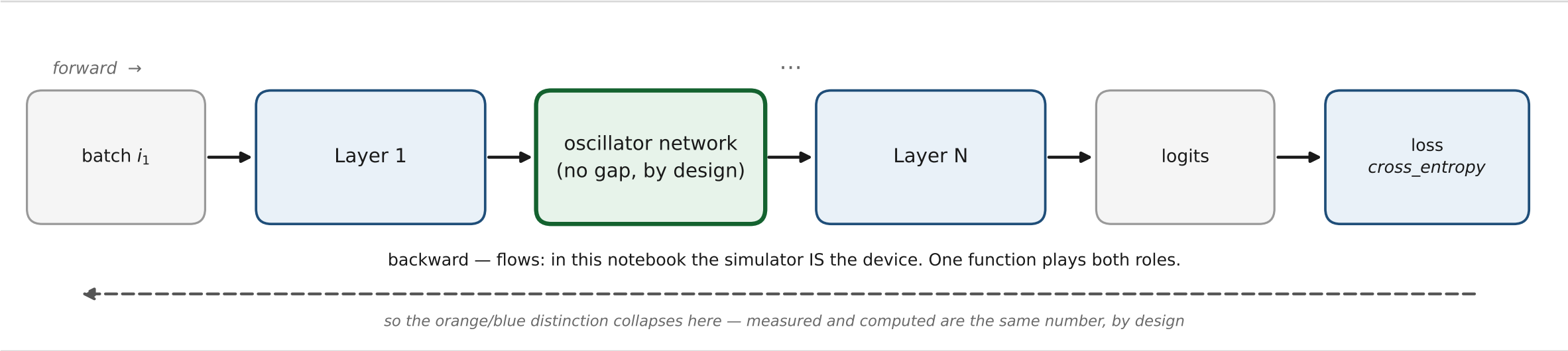
- a measured  $o_1$ , if you can get one, goes UNUSED
- gradient quality decays toward B's as the block deepens —  
the joints inside the block have no anchors
- one big simulator is harder to build than several small ones

△ VERDICT: legitimate — even necessary — when you cannot measure between layers. Wasteful when you can.

the spectrum: B (no real anchors) → E (anchored at the block's edges) → C (anchored at every joint)

the rule underneath PAT: wrap the physics at the finest granularity you can measure.

MNIST. The control: an oscillator network with NO reality gap — forward and backward are the same function. PAT is not used.



the physical layer, mapped onto layer vocabulary:

- input** pull each pendulum to a starting angle — the 14×14 image, loaded as angles
- weights** the spring stiffness between every pair, plus a steady push per pendulum — the knobs
- compute** let go and wait half a second — everything swings and tugs on everything else
- output** where the 10 designated “class” pendulums ended up — the logits
- activation** sin — nobody chose it, it falls out of gravity

✓ **RESULT: 98% on MNIST — with sin, a “terrible” activation. The only real requirement is: don't be linear.**

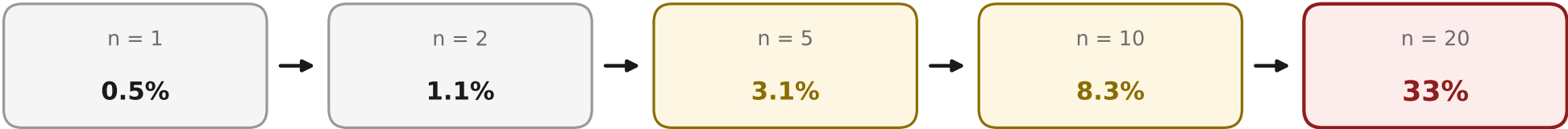
**what it proves: the architecture can learn — despite sin, symmetric weights, and “let time pass” as the only operation.**

why the no-gap setup is the point: it is the CONTROL. It clears the architecture as a suspect —

so when training fails in Experiments 3 and 2, only the GAP is left to blame. One variable at a time.

B and C, raced on a toy: reality is  $f(x) = 2.00 \cdot x^{1.1}$ , the simulator is  $2.01 \cdot x^{1.1}$  — 0.5% off, applied 20 times like 20 layers.

① the forward gap — reality and the simulator, drifting apart:



0.5% per layer compounds to 33% at n = 20 — every application works on the previous one's already-wrong answer

② now the GRADIENTS, same setup — how far off is each method at n = 20?

true gradient 0% — the reference. This is D's ideal, free here only because the “device” is a formula

B — in-silico 33% off — exactly as wrong as its forward pass

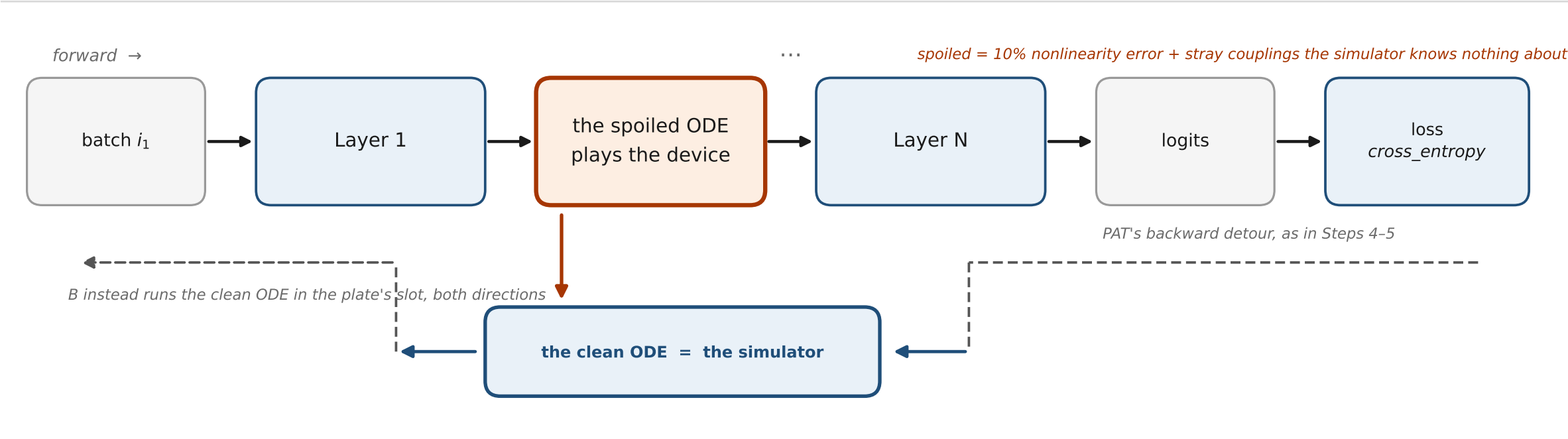
PAT 10.5% off =  $1.005^{20} - 1$  — ONLY the per-layer slope error. Nothing compounded.

PAT does not FIX the gradient — it stops the error compounding: exponential in depth → linear in depth.

✓ RESULT: with the same 0.5% modelling error, B's gradient is 33% wrong — PAT's stays at the irreducible 10.5% floor.

trained on a toy target, in-silico's loss then drifts away from reality while PAT tracks it — Experiment 2 repeats this at scale.

Fashion-MNIST. Same seed, same data order, same budget. The only difference is which function the forward pass runs through.



|                               | tested on the <b>DEVICE</b><br>what you actually get | tested on its own <b>SIMULATOR</b><br>what you would report |
|-------------------------------|------------------------------------------------------|-------------------------------------------------------------|
| <b>PAT</b>                    | <b>87.80%</b>                                        | <b>86.55%</b>                                               |
| <b>B — simulate both ways</b> | <b>79.25%</b>                                        | <b>87.55%</b>                                               |
|                               | ↑ reality: PAT wins by 8.55 points                   | ↑ in here, B looks <b>BETTER</b> — the inversion is total   |

✓ **RESULT: PAT's training-time numbers ARE its deployment numbers. B never warns you — it hands you 87.55% and ships 79.25%.**

bonus finding: PAT's weights score HIGHER on their own device (88.85%) than on the clean model (86.60%) — they absorbed that unit's defects.  
so trained weights belong to ONE physical machine: you ship the training procedure, not a weight file.

*the whole deck, one line per page.*

**A****freeze the physical layer**

half-works (reservoir computing) — but the knobs never learn, so the device is a random mixer, not a computer.

**B****train against the simulator**

0.5% of modelling error compounds to 33% over 20 layers — and it fails SILENTLY: everything looks excellent until deployment.

**C****PAT**

run REALITY forward; ask the simulator only for SLOPES, taken at reality's values. The error stops compounding.

**D****measure the gradient directly**

exact, but one hardware run per knob per step — unaffordable. This is why C borrows B's simulator for its slopes.

**E****wrap the whole device as one block**

works — real forward, one simulator — but measured intermediates go unused. Wrap at the finest granularity you can measure.

**1****MNIST — the control**

physics CAN learn: 98% with sin as the activation. Any nonlinearity works — “don't be linear” is the whole requirement.

**3****the toy function**

the argument in three numbers: forward gap 33%, B's gradient 33%, PAT's gradient 10.5% =  $1.005^{20} - 1$ .

**2****Fashion-MNIST**

PAT 87.80% vs B 79.25% on the device — while B reports 87.55% to itself. Ship the procedure, not the weights.

**an approximate gradient applied to a real measurement beats an exact gradient applied to an imaginary one.**

# Taxonomy — the vocabulary, one entry each

*come back here whenever a word stops making sense.*

## THE TRAINING LOOP

### logits

the network's raw output scores, one per class

### label

the dataset's human-written right answer — used only in the loss

### loss (cross\_entropy)

ONE number: how wrong the logits are against the label

### forward pass

data flowing through the layers to produce the logits

### backward pass

the gradient walking back from the loss, layer by layer

### gradient / slope

per knob: which direction, and how strongly, turning it would  
reduce the loss. A rate of change — never a value

### autograd

torch's bookkeeper — records every operation on the way forward,  
replays the recording backwards

### θ (theta)

all the trainable numbers, as floats on your computer.  
Physically: the device's dial settings

### optimizer.step()

the ONLY place θ ever changes — each knob nudged along its gradient

## ALSO SEEN

### in-silico

“in software” — trained entirely against the simulator: option B

### i<sub>1</sub>, o<sub>1</sub>, i<sub>2</sub>, o<sub>2</sub>

layer inputs and outputs. i<sub>2</sub> = o<sub>1</sub> is the JOINT;  
every o is MEASURED on the device, and saved as the next i

## THE PLAYERS

### the device / the plate (f\_exp)

the physical object playing a layer — you can run it and  
measure it; nothing else. In this repo: a spoiled ODE

### the simulator (f\_model / f\_backward)

code that predicts the device — built once, frozen, NOT part of  
the network; PAT uses only its slopes. The paper calls it the  
“differentiable digital model”

### the gap

the DISAGREEMENT between device and simulator. Not an object:  
zero in Ex1, a 0.5% coefficient in Ex3, deliberate spoilage in Ex2

### the joint

where one layer's output becomes the next layer's input — the only  
place error compounds. PAT's rule: only measurements cross joints

### saved inputs

the (x, θ) a physical layer received going forward, kept so the  
simulator's slope can be taken AT them

### vjp

vector-Jacobian product: run a function once, get its slope back.  
Returns (prediction, gradient); PAT discards the prediction

## THE PHYSICS

### ODE

an equation giving rates of change, never outcomes.  
“Integrating” = stepping it forward in small time slices —  
hardware does that for free; the simulator does it in code

### nonlinearity / activation

the bend between layers — without one, stacked layers collapse  
into a single matrix. ANY bend works; sin is fine